

# Kelas 11 Fase F - 05 Pemrograman Lanjut

## Rancangan Pembelajaran Terdiferensiasi — Fase F Kelas 11

### Materi: Algoritma dan Pemrograman Lanjut

#### 1. Tujuan Pembelajaran

- 1.1 Memahami konsep strategi algoritmik dalam pemrograman.
- 1.2 Mengembangkan program komputer terstruktur, modular, efektif, efisien, dan optimal.
- 1.3 Mengembangkan, melakukan pemeliharaan, dan penyempurnaan algoritma standar ke dalam kode sumber program.
- 1.4 Memperhatikan kualitas program (*readability, maintainability, efficiency*).
- 1.5 Merancang dan mengimplementasikan program dengan struktur data kompleks.
- 1.6 Menggunakan *library* atau perangkat yang tersedia, termasuk modul AI.
- 1.7 Mengintegrasikan program dengan perangkat lain (IoT, robot fisik, atau API).

#### 2. Kompetensi Prasyarat

| No | Kompetensi Prasyarat                                                  | Keterkaitan                                |
|----|-----------------------------------------------------------------------|--------------------------------------------|
| 1  | Peserta didik mampu menulis pseudocode dan program sederhana (Fase E) | Dasar pemrograman lanjut                   |
| 2  | Peserta didik memahami struktur data (list, stack, queue) (Fase E)    | Prasyarat struktur data kompleks           |
| 3  | Peserta didik memahami strategi algoritmik dasar (materi sebelumnya)  | Prasyarat implementasi strategi dalam kode |
| 4  | Peserta didik mampu menggunakan <i>spreadsheet/tools</i> digital      | Prasyarat eksplorasi <i>library</i>        |

#### 3. Asesmen Awal

##### 3.1 Instrumen Asesmen Awal

**Bentuk:** Praktik *coding* diagnostik + soal pemahaman konsep + *self-assessment*.

**Praktik Coding Diagnostik:** > Berikan soal berikut dalam bahasa Python (atau bahasa yang diajarkan): >> 1. Buat program yang menerima 3 angka dan mencetak angka terbesarnya! > 2. Buat program yang mencetak deret Fibonacci hingga suku ke-n! >

**Soal Konsep:** 1. “Apa perbedaan program terstruktur dan tidak terstruktur?” 2. “Apa yang dimaksud dengan *library/module* dalam pemrograman? Berikan contoh!” 3. “Bagaimana cara mengukur kualitas sebuah program?”

**Self-Assessment:** - [ ] Saya bisa menulis program dengan fungsi/prosedur - [ ] Saya pernah menggunakan *library* pihak ketiga - [ ] Saya paham konsep *array/list* multidimensi - [ ] Saya pernah membuat program yang terhubung ke *file/database* - [ ] Saya bisa membaca dan memperbaiki *error/bug*

##### 3.2 Kriteria Kesiapan Belajar

| Kondisi                                                                                  | Kategori    | Tindak Lanjut                                                                            |
|------------------------------------------------------------------------------------------|-------------|------------------------------------------------------------------------------------------|
| Kesulitan membuat program sederhana, belum memahami fungsi, <i>self-assessment</i> ≤ 1   | Kurang Siap | Review dasar pemrograman (variabel, percabangan, perulangan, fungsi sederhana)           |
| Mampu membuat program sederhana, memahami fungsi, <i>self-assessment</i> 2-3             | Cukup Siap  | Pengenalan modularisasi, <i>error handling</i> , dan penggunaan <i>library</i> dasar     |
| Mampu membuat program dengan fungsi + <i>error handling</i> , <i>self-assessment</i> ≥ 4 | Sudah Siap  | Struktur data kompleks, API/ <i>library</i> AI, integrasi IoT, kualitas kode profesional |

## 4. Rancangan Diferensiasi

### 4.1 Diferensiasi Konten

| Kelompok    | Sumber Belajar                                                                                                         | Kompleksitas                       |
|-------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Kurang Siap | Modul <i>review</i> dasar Python + kartu referensi sintaks + video tutorial fungsi + latihan bertahap                  | Review, bertahap, visual           |
| Cukup Siap  | Modul pemrograman modular + dokumentasi <i>library</i> standar + contoh proyek + panduan <i>debugging</i>              | Semi-kompleks, aplikatif           |
| Sudah Siap  | Dokumentasi <i>library</i> AI/ML + tutorial API + studi kasus program produksi + kode sumber proyek <i>open source</i> | Kompleks, profesional, eksploratif |

### 4.2 Diferensiasi Proses

| Kelompok    | Aktivitas Pembelajaran                                                                                                                   | Pendampingan                                     |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Kurang Siap | <i>Pair programming</i> dengan tutor: menulis program sederhana → ditambah fungsi → ditambah <i>error handling</i>                       | Pendampingan intensif, <i>driver-navigator</i>   |
| Cukup Siap  | Proyek individu/berpasangan: program modular dengan 3-4 fungsi + menggunakan 1 <i>library</i> eksternal + dokumentasi                    | <i>Code review</i> berkala, <i>scaffolding</i>   |
| Sudah Siap  | Proyek mandiri: program dengan struktur data kompleks + <i>library</i> AI/API + integrasi perangkat lain + pengujian + <i>deployment</i> | Konsultasi sesuai kebutuhan, standar profesional |

### 4.3 Diferensiasi Produk

| Kelompok    | Bentuk Produk                                                                                                                       | Kriteria                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Kurang Siap | Program kalkulator sederhana dengan fungsi (input, proses, output) + <i>error handling</i>                                          | Program berjalan, fungsi minimal 3, <i>handle error</i> dasar                        |
| Cukup Siap  | Program aplikasi dengan modularitas (min 5 fungsi) + menggunakan 1 <i>library</i> eksternal + dokumentasi                           | Modular, <i>library</i> eksternal, dokumentasi <i>readme</i> , <i>error handling</i> |
| Sudah Siap  | Program dengan struktur data kompleks + <i>library</i> AI/API + integrasi IoT/file/database + <i>unit test</i> + dokumentasi teknis | Kode produksi, <i>tested</i> , terintegrasi, dokumentasi lengkap                     |

## 5. Pengelompokan Fleksibel

- *Pair programming* dengan rotasi *driver* dan *navigator* setiap 15 menit.
- Kelompok proyek dapat dibentuk berdasarkan minat (aplikasi *desktop*, *web*, *data*, AI, IoT).
- *Code review* lintas kelompok: setiap kelompok meninjau kode kelompok lain dan memberikan umpan balik.

## 6. Asesmen Sumatif (Akhir Materi)

| Indikator                                | Perlu Perbaikan   | Cukup                       | Baik                                   | Sangat Baik                                    |
|------------------------------------------|-------------------|-----------------------------|----------------------------------------|------------------------------------------------|
| Program terstruktur                      | Monolitik         | 2-3 fungsi                  | 4-5 fungsi modular                     | Modular + <i>reusable</i>                      |
| Kualitas kode                            | Tidak rapi        | Rapi, tanpa komentar        | Rapi + komentar + <i>naming</i> baik   | Rapi + dokumentasi + <i>coding standard</i>    |
| Penggunaan <i>library</i>                | Tidak menggunakan | 1 <i>library</i>            | 2-3 <i>library</i>                     | <i>Library</i> + API/AI/IoT                    |
| <i>Debugging</i> & <i>error handling</i> | Tidak ada         | <i>Error handling</i> dasar | <i>Error handling</i> + <i>logging</i> | <i>Unit test</i> + <i>debugging</i> sistematis |
| Struktur data kompleks                   | <i>List</i> 1D    | <i>List</i> 2D/ <i>dict</i> | <i>Nested</i> struktur data            | Struktur data + algoritma optimal              |

| Indikator   | Perlu Perbaikan | Cukup      | Baik                    | Sangat Baik                |
|-------------|-----------------|------------|-------------------------|----------------------------|
| Dokumentasi | Tidak ada       | 1 paragraf | <i>Readme + comment</i> | Dokumentasi teknis lengkap |